

# Market-basket analysis

Alberto Polato

August 2026

## 1 Introduction

This project aims to implement a market-basket analysis, a technique originally used to find which items are frequently purchased together by people in a grocery store.

Here the market-basket analysis is used to find the pairs or triples of actors that are frequently present in movies, defining also association rules to answer the question: "If an actor A is present in a movie, how likely is Actor B to also be in that movie?".

To execute the algorithm efficiently and be able to scale up to millions of movies, Spark and the SON algorithm are used, described in more details in the algorithm and implementation section.

## 2 Data organization

The dataset used is the "Top 1000 Movies by IMDB Rating" dataset from Kaggle, which intuitively contains 1000 movies that are the baskets in our market-basket analysis. Each movie corresponds to a row containing data such as movie title, series title, release year and the variables that are interesting for our project, which are Star1, Star2, Star3 and Star4. These correspond to the four lead actors of the movie and are considered the items of the analysis (Figure 1.). The initial dataframe is clean to just contain the name of the actors and is transformed in an RDD, a resilient distributed dataset, used as it is an immutable collection that can be used in Spark.

This way the result is a row with the name of the actors, whose names are converted to string and removed of any whitespace before or after it. Then, in each row, the actors' names are alphabetically ordered.

## 3 Pre-processing techniques

During the creation of the baskets, each null value is removed from the actors column.

Since there is to analyze pairs, a basket has to contain at least two actors. After

this step, the baskets are still 1000, meaning that the data was already cleaned, even though this process is useful when changing the dataset.

Thresholds are defined so that we don't have too many frequent itemsets or too little, making the market-basket analysis not significant. The minimum support value is 0.004, so that an actor must appear in 0.4% of the movies to be considered frequent. Since, as said before, the number of movies is 1000, the actors need to appear in at least 4 movies to be considered frequent.

To make processing more efficient and to prevent the out of memory error, the dataset is divided into partitions to avoid uploading the whole dataset into memory. The number of partitions depend on the dataset and is a number between `sc.defaultParallelism*2` and `math.ceil(n_rows/target_rows_partition)`. In the first one, `defaultParallelism` matches the number of CPU cores available, so the number of partition is the CPU cores multiplied by 2, following the rule of thumb of Spark; the second one divides the total number of rows by 50000 and rounds up the number. This case is particularly useful when the code deals with massive datasets, since it has a fixed number of rows that can be in a partition. Else, if the code is going to use the first formula to divide the partition in a massive dataset, it would end up with too many rows for partition and an out of memory error.

## 4 Algorithm and implementation

Apriori is implemented to use the SON algorithm.

First, apriori is used for its monotonicity property, that says that if an itemset is frequent, than also all of its subsets must be frequent. This makes computation way more efficient, reducing the numbers of steps that the algorithm has to do to check if an itemset is frequent.

Candidate generation for size  $k$  joins the frequent sets of size  $k-1$ . Then, for every candidate, all  $k-1$  are checked against the already known frequent sets, so that if a subset is missing, then the candidate is removed.

SON algorithm works in two passes. In the first one, each partition or chunk runs the apriori with a scaled local support through the function `.mapPartitions`. The underlying theory behind this is based on the statement that any itemset that is globally frequent across the whole dataset has to be frequent in at least a partition. Than the union of these frequent itemsets brings to the candidate pool. In the IMDB dataset, this first phase produces 13488 candidates.

In the second pass, every basket in every partition is checked against the candidate pool of the first pass. Candidates that pass the global support test are considered frequent, and in the project the result of this phase is 148 frequent itemsets.

After the SON algorithm, the association rules are formed, using of course itemset of size  $k \geq 2$ . This rule are evaluated by two metrics, the confidence, measuring the conditional probability that the consequent appears given the antecedent; and the interest, which measures the difference between confidence and support of consequent.

## 5 Scalability

Spark provides the necessary scalability to be able to make the code function both on the IMDB dataset and on synthetic generated datasets with 2 millions movies. In particular, what makes the code scalable are the following factors, like the worker nodes in pass 1 that process independently the partitions they are assigned to; the memory fallback, thanks to the PySpark function `StorageLevel.MEMORY_AND_DISK`; and the mathematical guarantee that the SON algorithm doesn't discard frequent itemsets. As seen during the course, false negatives cannot exist in SON because  $s \leq \text{supp}(I) \leq \sum_C \text{supp}_C(I) < \sum_C ps = ps * 1/p = s$ . The result is  $s < s$ , which is not possible.

## 6 Description of experiments

To test the algorithm, first it was run over the IMDB dataset. Since it contains only 1000 movies, the runtime of the code was fast and there were no particular problems.

Other than the IMDB dataset, a synthetic dataset was used, containing 2 million movies with the same structure of the IMDB one. The probability of generating pairs or triples of frequent actors was set to 10% for this synthetic dataset, so that the it was sure that the algorithm was going to find a pattern. The `min_supp` variable value was increased to 0.05, to avoid too many frequent sets. The code is successfully able to run even in this conditions, with a runtime of about 5 minutes, which is considered reasonable since the scale of the dataset.

## 7 Results

The project is able to successfully create a market-basket analysis for the selected dataset. As shown in figure 2., the most frequent actor in the dataset is Robert De Niro, with 17 appearances over 1000 movies. In figure 3. we see that, when the pairs and triples are considered, things change quite a lot, since we can see that the ranking is dominated by actors of the Harry Potter saga. A thing that can seem odd is the fact that, despite having worked together in all the movies, Emma Watson appears in 5 movies, while Daniel Radcliffe and Rupert Grint in 6. This is due to the fact that in one of the movies, Emma Watson was not considered in the top 4 of the leading actors in the IMDB ranking.

The association rules brought 8 strong associations involving the Harry Potter cast and Tim Allen and Tom Hanks, that work together in the Toy Story franchise.

To better visualize these rules, a graph visualization was used and is shown in figure 4.

```

imdb_schema = StructType([
    StructField("Poster_Link", StringType(), True),
    StructField("Series_Title", StringType(), True),
    StructField("Released_Year", StringType(), True),
    StructField("Certificate", StringType(), True),
    StructField("Runtime", StringType(), True),
    StructField("Genre", StringType(), True),
    StructField("IMDB_Rating", DoubleType(), True),
    StructField("Overview", StringType(), True),
    StructField("Meta_score", DoubleType(), True),
    StructField("Director", StringType(), True),
    StructField("Star1", StringType(), True),
    StructField("Star2", StringType(), True),
    StructField("Star3", StringType(), True),
    StructField("Star4", StringType(), True),
    StructField("No_of_Votes", IntegerType(), True),
    StructField("Gross", StringType(), True),
])

```

Figure 1: Dataset structure

```

('Robert De Niro',) -> 17
('Tom Hanks',) -> 14
('Al Pacino',) -> 13
('Brad Pitt',) -> 12
('Clint Eastwood',) -> 12

```

Figure 2: Top single actors

## 8 Bibliography

Leskovec, Jure & Rajaraman, Anand & Ullman, Jeffrey. (2020). Mining of Massive Datasets. 10.1017/9781108684163.

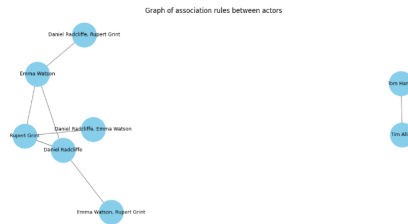
## 9 Appendix

```

[pair] ('Daniel Radcliffe', 'Rupert Grint') -> 6 movies
[pair] ('Daniel Radcliffe', 'Emma Watson') -> 5 movies
[triple] ('Daniel Radcliffe', 'Emma Watson', 'Rupert Grint') -> 5 movies
[pair] ('Emma Watson', 'Rupert Grint') -> 5 movies
[pair] ('Joe Pesci', 'Robert De Niro') -> 4 movies
[pair] ('Tim Allen', 'Tom Hanks') -> 4 movies

```

Figure 3: Top pairs and triples



## 10 Declaration

"I/We declare that this material, which I/We now submit for assessment, is entirely my/our own work and has not been taken from the work of others save and to the extent that such work has been cited and acknowledged within the text of my/our work, and including any code produced using generative AI systems. I/We understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me/us or any other person for assessment on this or any other course of study."